



TANTÁRGYAK

TANANYAGOK

PROJEKTEK

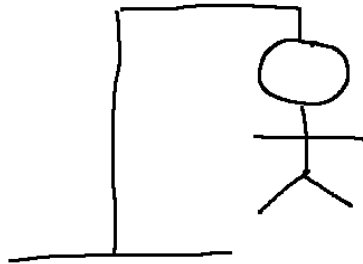
EBR

PRACTICE 04. – WEB PROGRAMMING

1. A treasure is hidden in a map represented by a square grid. By clicking on the cells, find the treasure! After finding it, the application will hide the treasure again!
 - a. The table should be 2x2 in size!
 - b. In a text box you can enter the size of the board. By clicking a button, the corresponding table is displayed.
 - c. Let's make a counter that measures how many clicks we have found the treasure!
2. Create a hangman game! The main features are:
 - Choose a random word! This can be baked in initially, or it can come as a random item from a list of words.
 - Display as many small lines on the user interface (UI) as the number of letters of the word!
 - Display the letters on the alphabet as buttons on the UI!
 - Let's have a section showing how many bad letters we've guessed so far and how many mistakes we can have in total!
 - Prepare some elements of the hangman game to be drawn using SVG elements.

_ B _ _ _

A B C D E ...



You can find the HTML and CSS boilerplate codes below. Analyze the code, try out the style classes prepared in CSS statically, you can find them as comments in the HTML file.

- The letters of the word are contained in the element with the identifier `the-word`. If the player wins, it gets the `won` style class! If the player loses, the spans of the unrevealed letters get the `missing` style class!
- The win/lose caption is in the element with `end-of-game` ID. It is hidden at first, but must be revealed with the appropriate caption text at the end of the game! Hide it again in a new game!
- The item with the ID `letters` contains the guessing buttons. After guessing, the button becomes `disabled`.
- The elements of the gallows are hidden in an `svg` element. As a test, we can see what it looks like if we add the `revealed` style class. Each element can be revealed by adding the `show` style class to it.

Additional tasks:

- Try to find and use a dictionary file to guess the random word.
- Indicate on the interface with a caption whether the player has won or not and there should be a "Play again!" button you can click to restart the game!
- Try to encapsulate the game logic as a JavaScript module!
- Try to encapsulate the logic of the interface as a separate module, too!

HTML:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <link rel="stylesheet" href="index.css" />
    <title>Hangman</title>
  </head>
  <body>
    <h1>Hangman game</h1>

    <!-- <div id="the-word" class="won"> -->
    <div id="the-word">
      <span>a</span>
      <span>b</span>
      <span></span>
      <span class="missing">b</span>
    </div>

    <!-- <div id="end-of-game"> -->
    <div id="end-of-game" hidden>
      <span>You win! / You lose!</span>
      <button>Play again!</button>
    </div>

    <div id="letters">
      <button>a</button>
      <button disabled>b</button>
      <button>c</button>
      <button>d</button>
    </div>

    <div id="score">Score: 1/9</div>

    <!-- <svg width="200px" height="200px" stroke="black" class="
  <svg width="200px" height="200px" stroke="black">
    <line x1="0" y1="99%" x2="100%" y2="99%" />
    <line x1="20%" y1="99%" x2="20%" y2="5%" />
    <line x1="20%" y1="5%" x2="60%" y2="5%" />
    <!-- <line x1="20%" y1="5%" x2="60%" y2="5%" class="show"
    <line x1="60%" y1="5%" x2="60%" y2="20%" />
    <circle cx="60%" cy="30%" r="10%" />
    <line x1="60%" y1="30%" x2="60%" y2="70%" />
    <line x1="40%" y1="50%" x2="80%" y2="50%" />
    <line x1="60%" y1="70%" x2="50%" y2="90%" />
    <line x1="60%" y1="70%" x2="70%" y2="90%" />
```

```
    </svg>
  </body>
</html>
```

CSS:

```
div {
  margin: 20px;
}
#the-word {
  display: flex;
}
#the-word span {
  width: 20px;
  height: 20px;
  border-bottom: 2px solid black;
  margin-left: 3px;
  text-align: center;
  font-size: 20px;
}
#the-word.won {
  color: green;
}
#the-word .missing {
  color: red;
}
#letters button[disabled] {
  opacity: 0.5;
}
svg {
  border: 10px solid lightgray;
}
svg:not(.revealed) * {
  display: none;
}
svg .show {
  display: block;
  animation: flash 2s 0.2s;
}
@keyframes flash {
  0% {
    stroke: red;
    stroke-width: 5px;
  }
  100% {
    stroke: black;
    stroke-width: 1px;
  }
}

#end-of-game {
```

```
border: 5px solid gold;  
font-size: x-large;  
padding: 10px;  
}
```